MEETING INTELLIGENCE ASSISTANT

From a recorded meeting to a verified, structured record

Team Linear Regression Akansha Das · Atrijo Pal · Anuj Sharma

Inter IIT Bootcamp, ML Problem Statement, Phase 2 · Source: github.com/atrijopal/boot

1 What the system does

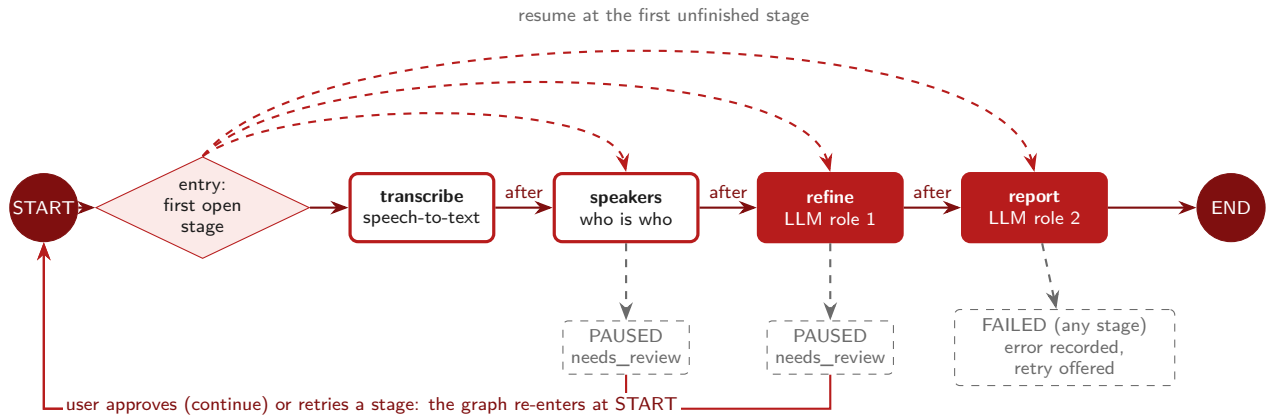
The application takes an English meeting recording and, in one run, produces the five outputs the problem statement requires: a **raw transcript**, a **refined transcript**, **meeting minutes**, a list of **key decisions** and a list of **action items**. Raw and refined transcripts are kept separately and compared in the interface. Every statement in the record is traceable to a quote, a speaker and a time in the recording. Where the meeting did not state an owner or a deadline, the record says *unspecified*; a proposal is never reported as a decision, and an unstated assignment is never reported as a confirmed task.

Output	Content (from the brief)	Where it is produced and exported
Raw transcript	Speech-to-text result before language-model refinement	Transcript tab, TXT and JSON
Refined transcript	After domain-aware terminology correction; every change listed	Refinement tab, TXT and JSON
Meeting minutes	Concise summary and organised account of the discussion	Report: summary, topics. PDF, Markdown, HTML
Key decisions	Decisions reached; an empty list if none	Report and record JSON (`decisions`)
Action items	Task, stated owner or deadline, otherwise *unspecified*	Report, record JSON, CSV, ICS calendar

The human-readable record (PDF, Markdown, HTML) and the machine-readable record (JSON, schema `meeting-record/1`) are generated from one structure, so both carry identical decisions and tasks; this is enforced by tests.

2 Architecture

The pipeline is a **LangGraph** state machine with four nodes in the order the brief prescribes: transcription, speaker identification, refinement (language-model role 1) and documentation (language-model role 2). The two language-model roles are separate stages with separate prompts, models and outputs. The backend owns the state: each meeting has a persisted session file with an explicit state per stage (`waiting`, `processing`, `needs_review`, `completed`, `failed`). The interface only reads it. Two stages end in a human checkpoint (speakers, refinement) where the graph halts and resumes when the user approves; a failed stage can be retried alone, keeping earlier work.



session.json: persisted state of all five stages, written atomically before and after every node

Figure 1: LangGraph state machine. Solid arrows are the normal flow; dashed arrows are conditional routes decided from the persisted session.

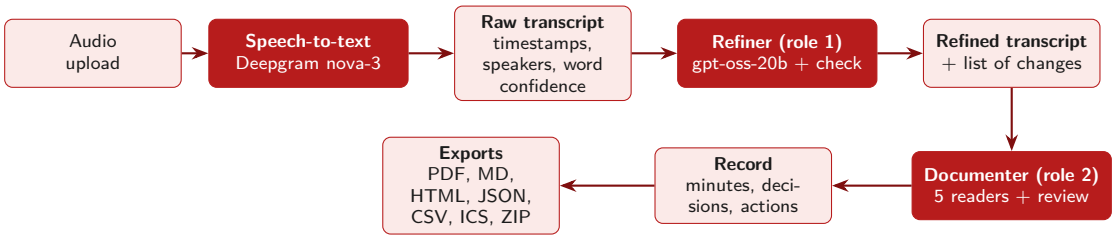


Figure 2: How outputs move between stages. The documenter reads the *refined* transcript, never the raw one.

3 Models and fallbacks

Role	Model	Purpose
Speech-to-text	Deepgram **nova-3**	Whole file in one request; word timestamps, confidences and *voice-based* speaker labels
Speech-to-text (alt.)	Groq **whisper-large-v3**, 10-min chunks	Used when no Deepgram key is set; text only, no speakers
Role 1: proposer	**gpt-oss-20b** (reasoning: medium)	Proposes small exact fixes to terms, acronyms, names
Role 1: second check	**qwen3.8-27b** (low)	Different model family confirms or rejects a fix that has no support in the recording
Speaker names	**gpt-oss-20b** (low)	Suggests a name or role only with a quote that shows it
Role 2: readers	**qwen3.8-27b** $\times 4$, **gpt-oss-120b** $\times 1$	Five focused readers (see below); the 120b reads tasks, owners and deadlines, the error-prone part
Role 2: review	**gpt-oss-120b** (low)	Checks decisions and tasks, writes headline, summary, topic bullets

Why several models. Each call uses the smallest model and reasoning effort that does its job, and the review model is a different family from most readers, so it checks them independently rather than repeating their mistakes.

Fallbacks. (1) A language-model call that is refused or unavailable is answered by a configured backup model (`FALLBACK_MODEL`, NVIDIA nemotron-3-super-120b); both attempts remain visible in the audit view. (2) A speech-to-text chunk falls back from whisper-large-v3 to whisper-large-v3-turbo; the provider is chosen automatically (Deepgram if configured). (3) An unreadable JSON answer is retried, once with lower reasoning effort. (4) If the review cannot run, the report is still produced from the verified records, the summary is restated from verbatim quotes, and every decision is tagged *not confirmed* with a banner and a "run the final check again" action. (5) A failed stage is retried alone; finished stages are kept.

4 Prompts

All 13 prompts are stored as files in `prompts/` and are the only model instructions. Each treats the transcript as delimited data (a prompt-injection firewall), demands JSON only, and states what must *not* be done.

Prompt	Stage	Instruction in brief
`refine`	Refinement	Return edits only (`find`, `rep`, kind), never rewritten text; fix terms, acronyms, names, spelling; never touch numbers, dates, negations or commitments
`verify_edits`	Refinement	Same meaning? Anything new added? Reject if unsure
`names`	Speakers	Name or role only from self-introduction or being addressed and answering; quote required
`speakers`	Speakers	Optional text-only speaker labelling (off by default; measured unreliable)
`extract_decisions`	Report	Decisions and proposals; a proposal is not a decision
`extract_actions`	Report	Tasks; owner only if the speaker commits in first person or is named; deadline only if quoted
`extract_issues`	Report	Questions, concerns, risks, disagreements
`extract_details`	Report	Figures, topics, names
`extract_discussion`	Report	Main discussion points
`extract_core`, `extract_context`, `extract`	Report	Two-reader and single-reader variants used when a long meeting must fit a call budget

Prompt	Stage	Instruction in brief
`review`	Report	Verdicts that can only remove or weaken a record; headline, summary citing record ids, topic bullets, per-speaker summaries

Safeguards implemented in code (no model involved)

- A quote must appear *word for word* in the transcript; a quote may not skip a negation; a number in a record must be in its quote.
- A correction may not touch a number, date, time, duration or negation, may not overlap them, and is size-limited; the check re-runs the extractors on the edited text.
- A correction is applied automatically only if the recording supports it (same word elsewhere, the recogniser's own alternative) or the independent second check confirms it; otherwise it is only *suggested*.
- Owner: first-person commitment by a labelled speaker, or named in the quote; deadline: only if quoted; otherwise *unspecified*.
- Every summary sentence must cite kept records and add no new number; duplicate records from overlapping windows are merged.
- The server re-validates every correction a user accepts; speaker names are shown only when the meeting shows them or the user types them.

5 Token efficiency

1. **Compact encoding:** transcript lines as `u012 A: text`, short consecutive lines of one speaker merged, short JSON keys.
2. **Code does the cheap work:** numbers, dates, durations, negations, acronyms and rare words are found by code and pre-flagged, so the model does not hunt for them.
3. **Edit-only refinement:** the model returns a few `find`/`rep` pairs, not the transcript.
4. **The review reads records, not the transcript** (about 4k tokens instead of the full text).
5. **Windowing:** about 3,500 transcript tokens per call; a hard call cap (6) makes windows grow or readers merge rather than the cost grow.
6. **Right-sized models and effort:** low effort for narrow tasks; the 120b only for owners/deadlines and the review.
7. **Second check only when needed:** only for suggested fixes without evidence in the recording.
8. **Budgets:** 7,000 tokens per call and 80,000 per run; a run that would exceed them is refused with a clear message before it starts.
9. **Content-addressed cache and stage retry:** identical inputs cost zero tokens; a retry re-runs only the failed stage.

Measured on a 5-minute meeting (full run, all stages completed):

Stage	Calls	Tokens	Model(s)	Share
Speaker names	1	1,757	gpt-oss-20b	8%
Refinement (+ check)	1	3,531	gpt-oss-20b / qwen	16%
Five readers	5	11,873	qwen ×4, 120b	55%
Review and summary	1	4,270	gpt-oss-120b	20%
Total	**8**	**21,431**	20b 25%, qwen 41%, 120b 35%	

That is about **4.3k tokens per minute of audio** and a mean of about 2.7k tokens per call. The readers scale with length (17.1k for a 7.5-minute meeting against 11.9k for 5 minutes) while the review stays near 4–6k. Splitting extraction into focused readers raised the number of verified records on the same 12-minute clip from 3–5 to 12–13 for roughly 4–5k extra tokens.

6 Interface

The interface follows the workflow: Upload, Speakers, Transcript, Refinement, Meeting report, Export, plus a Pipeline tab that shows what each stage received and returned. A stage bar shows status at all times, with plain-language errors and a retry button for a failed stage.

- **Upload:** WAV, MP3, M4A, FLAC, OGG, OPUS, WebM, AAC; unsupported, empty, silent, corrupt or too-short files are rejected with an explanation.
- **Speakers:** suggested names with their evidence quote; accept, edit, or leave a speaker unnamed.
- **Transcript:** raw and refined views, search with highlighting, speaker filter, click a timestamp to play the real recording from that point, copy and download.

- **Refinement review:** original, proposed text, reason and evidence for every change; accept, reject, accept all, filters, side-by-side view.
- **Report:** a numbered document; every item links to its line in the transcript; *Unspecified* for missing owners and deadlines; uncertainty listed with severity (info, review, warning, error).
- **Export:** only files that exist are offered, with professional names; a ZIP bundles them with a manifest.

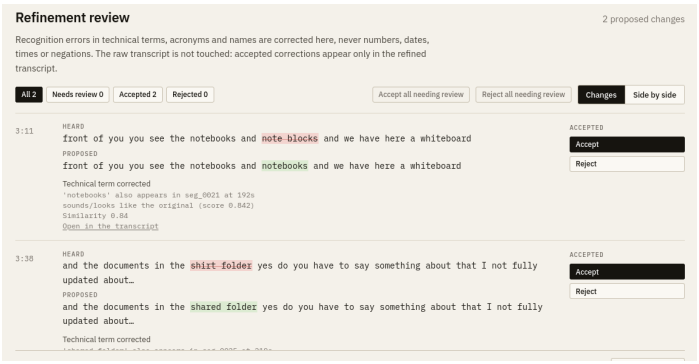


Figure 3: Refinement review: each proposed change with its evidence.

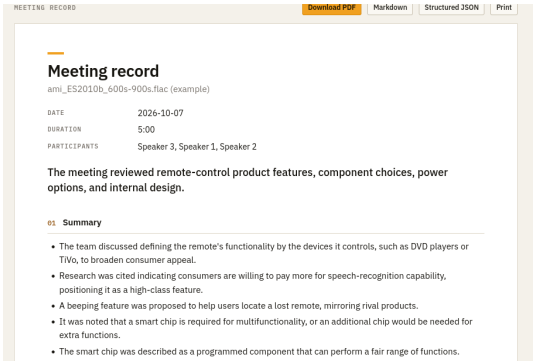


Figure 4: Meeting record: every item carries its source.

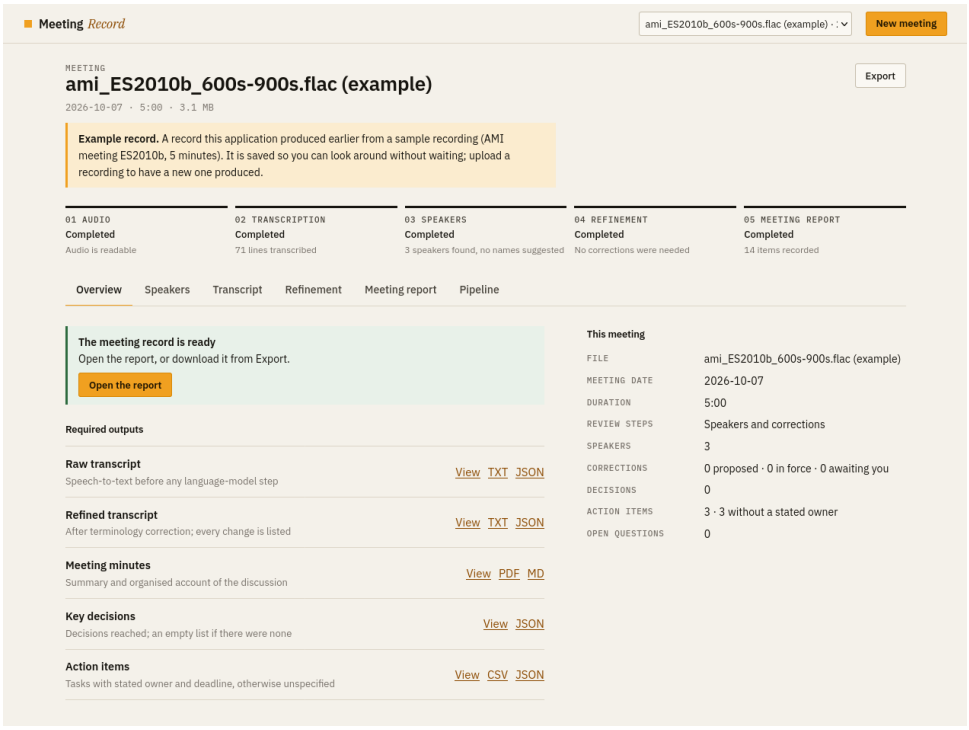


Figure 5: Overview: stage status, next step, and the five required outputs, each with a view link and downloads.

7 Evaluation

Ground truth comes from the public human transcripts and annotations of the AMI Meeting Corpus (CC BY 4.0); the repository provides scripts to build a reference for any clip and score a run (`scripts/make_reference.py`, `scripts/evaluate.py`).

Measurement (12-minute clips, single runs)	Result	Reference
Word error rate, Deepgram nova-3, quiet meeting (ES2002a)	19.8%	human transcript
Word error rate, Deepgram nova-3, loud office meeting (IB4001)	25.4%	human transcript
Word error rate, Groq whisper-large-v3, same office clip	29.8%	human transcript
Words given the correct speaker (4 real speakers found)	95.7% / 86.5%	human labels
Speaker labels guessed from text alone	28–40%	chance level; switched off

Most word errors are words lost where people talk over each other. Refinement was checked with planted terminology errors: 8 of 9 were corrected, and an earlier false correction of a person's name was removed by the stricter prompt and the second check. The suite contains **174 automated tests** that run offline with a faked recogniser and model:

the gates, graph routing, session recovery, every export, error messages, and a scan proving no model name, token count or prompt text reaches a user-visible page or file.

8 Deliverables and limits

The repository contains the full application, all prompts, dependency list and a README with setup and run steps (`bash run.sh`); sample recordings with reference transcripts; and generated example outputs (raw and refined transcript, minutes, decisions, action items) in `samples/example_outputs/`. A saved example meeting is labelled as such and is never returned for a new upload: every new recording is processed by the pipeline.

Limits. English only. Language models can be wrong; the checks reduce this but do not prove correctness. Speaker separation needs the Deepgram provider. Long meetings beyond about 15 minutes have been exercised by tests with a fake model, not live. Source: github.com/atrijopal/boot.